

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по рубежному контролю №2
«Модульное тестирование»

Выполнил:
Студент группы ИУ5-31Б
Рыбин Матвей

Проверил:
Гапанюк Ю. Е.

2025 г.

Листинг программы

```
from operator import itemgetter

class Book:
    def __init__(self, id, title, price, shop_id):
        self.id = id
        self.title = title
        self.price = price
        self.shop_id = shop_id

class BookShop:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class BookShopLink:
    def __init__(self, shop_id, book_id):
        self.shop_id = shop_id
        self.book_id = book_id

shops = [
    BookShop(1, "Читай-город"),
    BookShop(2, "Лабиринт"),
    BookShop(3, "Книжный дом"),
    BookShop(4, "Дом книги"),
    BookShop(5, "Буквоед")
]

books = [
    Book(1, "Преступление и наказание", 500, 1),
    Book(2, "Мастер и Маргарита", 650, 2),
    Book(3, "Война и мир", 800, 3),
    Book(4, "Отцы и дети", 450, 3),
    Book(5, "Евгений Онегин", 400, 3),
    Book(6, "Герой нашего времени", 550, 1)
]

book_shop_links = [
    BookShopLink(1, 1),
    BookShopLink(2, 2),
    BookShopLink(3, 3),
    BookShopLink(3, 4),
    BookShopLink(3, 5),
    BookShopLink(1, 6),
    BookShopLink(4, 1),
    BookShopLink(4, 3),
    BookShopLink(5, 2),
    BookShopLink(5, 5)
]

def get_one_to_many(shops, books):
    return [(b.title, b.price, s.name)
            for s in shops
            for b in books
            if b.shop_id == s.id]

def get_many_to_many(shops, books, book_shop_links):
    many_to_many_temp = [(s.name, bs.shop_id, bs.book_id)
                          for s in shops
                          for bs in book_shop_links
                          if s.id == bs.shop_id]

    return [(b.title, b.price, shop_name)
            for shop_name, shop_id, book_id in many_to_many_temp
            for b in books if b.id == book_id]
```

```

def task_a1(one_to_many):
    print('Задание A1')
    print('Список всех связанных книг и магазинов (отсортировано по магазинам):')
    res_1 = sorted(one_to_many, key=itemgetter(2))
    for item in res_1:
        print(f' Книга: {item[0]}, Цена: {item[1]}, Магазин: {item[2]}')

def task_a2(shops, one_to_many):
    print('\nЗадание A2')
    print('Список магазинов с суммарной стоимостью книг:')
    res_2_unsorted = []
    for s in shops:
        s_books = list(filter(lambda i: i[2] == s.name, one_to_many))
        if len(s_books) > 0:
            s_prices = [price for _, price, _ in s_books]
            s_prices_sum = sum(s_prices)
            res_2_unsorted.append((s.name, s_prices_sum))

    res_2 = sorted(res_2_unsorted, key=itemgetter(1), reverse=True)
    for item in res_2:
        print(f' Магазин: {item[0]}, Суммарная стоимость: {item[1]}')

def task_a3(shops, many_to_many):
    print('\nЗадание A3')
    print('Список магазинов с "дом" в названии и их книги:')
    res_3 = {}
    for s in shops:
        if 'дом' in s.name.lower():
            s_books = list(filter(lambda i: i[2] == s.name, many_to_many))
            s_book_titles = [title for title, _, _ in s_books]
            res_3[s.name] = s_book_titles

    for shop_name, book_list in res_3.items():
        print(f' Магазин: {shop_name}')
        for book in book_list:
            print(f' Книга: {book}')

def main():
    one_to_many = get_one_to_many(shops, books)
    many_to_many = get_many_to_many(shops, books, book_shop_links)

    task_a1(one_to_many)
    task_a2(shops, one_to_many)
    task_a3(shops, many_to_many)

if __name__ == '__main__':
    main()

```

```

import unittest
from book_shop import Book, BookShop, BookShopLink, get_one_to_many, get_many_to_many,
task_a1, task_a2, task_a3

shops = [
    BookShop(1, "Читай-город"),
    BookShop(2, "Лабиринт"),
    BookShop(3, "Книжный дом"),
    BookShop(4, "Дом книги"),
    BookShop(5, "Буквоед")
]

books = [
    Book(1, "Преступление и наказание", 500, 1),
    Book(2, "Мастер и Маргарита", 650, 2),
    Book(3, "Война и мир", 800, 3),
    Book(4, "Отцы и дети", 450, 3),
    Book(5, "Евгений Онегин", 400, 3),
    Book(6, "Герой нашего времени", 550, 1)
]

book_shop_links = [
    BookShopLink(1, 1),
    BookShopLink(2, 2),
    BookShopLink(3, 3),
    BookShopLink(3, 4),
    BookShopLink(3, 5),
    BookShopLink(1, 6),
    BookShopLink(4, 1),
    BookShopLink(4, 3),
    BookShopLink(5, 2),
    BookShopLink(5, 5)
]

class TestBookShop(unittest.TestCase):
    def setUp(self):
        self.one_to_many = get_one_to_many(shops, books)
        self.many_to_many = get_many_to_many(shops, books, book_shop_links)

    def test_one_to_many_length(self):
        self.assertEqual(len(self.one_to_many), 6)

    def test_many_to_many_length(self):
        self.assertEqual(len(self.many_to_many), 10)

    def test_one_to_many_contains_correct_shop(self):
        shop_names = [item[2] for item in self.one_to_many]
        self.assertIn("Читай-город", shop_names)
        self.assertIn("Книжный дом", shop_names)

    def test_task_a2_calculation(self):
        from book_shop import task_a2
        one_to_many = get_one_to_many(shops, books)
        res = []
        for s in shops:
            s_books = list(filter(lambda i: i[2] == s.name, one_to_many))
            if len(s_books) > 0:
                s_prices = [price for _, price, _ in s_books]
                s_prices_sum = sum(s_prices)
                res.append((s.name, s_prices_sum))
        for name, total in res:
            if name == "Книжный дом":
                self.assertEqual(total, 1650)
                break

    def test_task_a3_filter(self):
        from book_shop import task_a3
        result = {}

```

Результат выполнения:

